

计算流体力学作业 4

郑恒 2200011086

2025 年 5 月 1 日

1 问题介绍

本次作业研究一个矩形平板的稳态温度场分布问题。平板尺寸为 $15\text{ cm} \times 12\text{ cm}$ ，假设板的表面是绝热的，仅在四条边有热流通过。一条 15 cm 的边温度保持为 100°C ，其余三条边温度为 20°C 。

为求解该问题，采用离散代数方法，使用 SOR 超松弛高斯赛德尔迭代方法计算温度场，并绘制等温线。具体步骤如下：

1. 将矩形平板离散为网格点，建立离散方程；
2. 选择不同的松弛因子 ω ，比较收敛速度；
3. 采用不同的网格尺寸，观察最佳松弛因子的变化；
4. 使用 Python 编程实现计算，并绘制温度场的等温线图。

2 算法原理

二维稳态温度场的拉普拉斯方程为：

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0$$

将矩形平板离散为 $N_x \times N_y$ 的网格，网格点间距为 Δx 和 Δy ，离散化后得到：

$$\frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{\Delta x^2} + \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{\Delta y^2} = 0$$

整理得（高斯-赛德尔迭代公式）：

$$T_{i,j}^{(k+1)} = \frac{\Delta y^2(T_{i+1,j}^{(k)} + T_{i-1,j}^{(k+1)}) + \Delta x^2(T_{i,j+1}^{(k)} + T_{i,j-1}^{(k+1)})}{2(\Delta x^2 + \Delta y^2)}$$

为加速迭代过程，采用 SOR（Successive Over-Relaxation）方法，更新公式为：

$$T_{i,j}^{(k+1)} = (1-\omega)T_{i,j}^{(k)} + \frac{\omega}{2(\Delta x^2 + \Delta y^2)} \left[\Delta y^2(T_{i+1,j}^{(k)} + T_{i-1,j}^{(k+1)}) + \Delta x^2(T_{i,j+1}^{(k)} + T_{i,j-1}^{(k+1)}) \right]$$

其中， ω 为松弛因子， $0 < \omega < 2$ 。通过调整 ω ，可以优化收敛速度。

在本节中，我们假设 Δx 和 Δy 的大小相等，即：

$$\Delta x = \Delta y$$

这一假设在后续计算中将被广泛使用。

代入之前的公式有：

$$T_{i,j}^{(k+1)} = \frac{T_{i+1,j}^{(k)} + T_{i-1,j}^{(k)} + T_{i,j+1}^{(k)} + T_{i,j-1}^{(k)}}{4}$$

3 代码实现

代码框架如下所示：

首先是核心求解函数：

```
1 class ThermalSolver:
2     def __init__(self, Lx=15, Ly=12):
3         self.Lx = Lx # x方向物理尺寸(cm)
4         self.Ly = Ly # y方向物理尺寸(cm)
5
6     def solve(self, m=50, n=40, omega=1.0, tol=1e-3,
7               max_iter=5000):
8         """求解二维稳态温度场"""
9         T = np.zeros((m, n))
10
11         # 设置边界条件
```

```

11     T[:, -1] = 100 # 上边界
12     T[0, :] = 20 # 左边界
13     T[-1, :] = 20 # 右边界
14     T[:, 0] = 20 # 下边界
15
16     residuals = []
17     for _ in range(max_iter):
18         max_err = 0
19         # 更新内部节点
20         for i in range(1, m - 1):
21             for j in range(1, n - 1):
22                 old = T[i, j]
23                 new = (T[i + 1, j] + T[i - 1, j] + T[i,
24                     j + 1] + T[i, j - 1]) / 4
25                 T[i, j] = (1 - omega) * old + omega *
26                     new
27                 max_err = max(max_err, abs(T[i, j] -
28                     old))
29
30         residuals.append(max_err)
31         if max_err < tol:
32             break
33
34     return T, residuals

```

Listing 1: Python 代码实现

迭代终止条件为: $|T_{\text{new}} - T_{\text{old}}| < \text{tol}$

下根据不同的网格尺寸和松弛因子进行迭代, 计算温度场分布的迭代次数, 以此判断迭代速率。

```

1 # ===== 主程序 =====
2
3 if __name__ == "__main__":
4     solver = ThermalSolver()

```

```

5
6 # 调整网格配置参数
7 grid_configs = [
8     {'m': 5, 'n': 4, 'label': '5x4 (Coarsest)', 'color'
9         : 'blue',
10        'omega_range': (0.5, 1.6), 'tol': 1e-2},
11    {'m': 15, 'n': 12, 'label': '15x12 (Medium)', '
12        color': 'green',
13        'omega_range': (1.0, 1.8), 'tol': 1e-3},
14    {'m': 30, 'n': 24, 'label': '30x24 (Fine)', 'color'
15        : 'red',
16        'omega_range': (1.2, 1.9), 'tol': 1e-3},
17    {'m': 50, 'n': 40, 'label': '50x40 (Finest)', '
18        color': 'purple',

```

Listing 2: 不同大小的网格尺寸和松弛因子

以此画出不同网格尺寸和松弛因子下的迭代次数的关系图:

```

1 plt.figure(figsize=(14, 8))
2
3 for config in grid_configs:
4     # 参数提取
5     m = config['m']
6     n = config['n']
7     label = config['label']
8     color = config['color']
9     omega_start, omega_end = config['omega_range']
10    tol = config['tol'] # 不同网格使用不同收敛标准
11
12    # 生成omega范围
13    omega_range = np.linspace(omega_start, omega_end,
14                               30)
15
16    iterations = []

```

```

16     valid_omegas = []
17
18     for omega in omega_range:
19         # 调整求解参数
20         _, res = solver.solve(m=m, n=n, omega=omega,
21                               tol=tol, # 使用网格专属
22                                       收敛标准
23                                       max_iter=10000)
24
25         # 收敛判断条件
26         if len(res) < 10000 and res[-1] < tol:
27             iterations.append(len(res))
28             valid_omegas.append(omega)
29
30     # 仅在有数据时绘图
31     if len(valid_omegas) > 0:
32         plt.semilogy(valid_omegas, iterations, 's-',
33                      markersize=8,
34                      linewidth=2,
35                      color=color,
36                      label=label)
37
38     # 标注最佳点
39     try:
40         best_idx = np.argmin(iterations)
41         plt.annotate(f' = {valid_omegas[best_idx]:.2'
42                     f'}\n{iterations[best_idx]}',
43                     (valid_omegas[best_idx],
44                     iterations[best_idx]),
45                     textcoords="offset points",
46                     xytext=(10, -20),
47                     ha='left',
48                     arrowprops=dict(arrowstyle="->

```

```

46         ", color=color),
47         color=color)
48     except Exception as e:
49         print(f"标注失败: {label} - {str(e)}")
50     else:
51         print(f"警告: {label} 无有效数据")
52
53     # 图表参数调整
54     plt.xlabel('Relaxation Factor ', fontsize=12)
55     plt.ylabel('Iterations (log scale)', fontsize=12)
56     plt.title('Optimized Relaxation Factor Analysis',
57             fontsize=14)
58     plt.grid(True, which='both', alpha=0.3)
59     plt.legend(loc='upper left')
60     plt.xlim(0.4, 2.0)
61     plt.ylim(5, 10000)
62     plt.tight_layout()
63     plt.savefig('convergence_analysis.jpg',
64             dpi=300,
65             bbox_inches='tight')
66     plt.show()

```

Listing 3: 不同网格尺寸和松弛因子下的迭代次数的关系图

再根据最佳的松弛因子和网格尺寸进行迭代，计算温度场分布，并绘制等温线图：

```

1     # 最密网格可视化
2     plt.figure(figsize=(12, 8))
3     T_fine, _ = solver.solve(m=50, n=40, omega=1.87)
4
5     X, Y = np.meshgrid(np.linspace(0, 15, 50),
6                       np.linspace(0, 12, 40))
7
8     levels = np.arange(20, 101, 5)

```

```

9      contour = plt.contour(X, Y, T_fine.T, levels=levels,
      colors='white', linewidths=0.5)
10     plt.clabel(contour, inline=True, fmt='%d°C', fontsize=8)
11
12     im = plt.imshow(T_fine.T, extent=[0, 15, 0, 12], origin
      = 'lower',
13                      cmap='inferno', alpha=0.9)
14     plt.colorbar(im, label='Temperature (°C)', shrink=0.8)
15
16     plt.title('Temperature Field (50x40 Grid)', fontsize
      =14)
17     plt.xlabel('x (cm)')
18     plt.ylabel('y (cm)')
19     plt.tight_layout()
20     plt.savefig('temperature_field.jpg',
21                dpi=300,
22                bbox_inches='tight')
23     plt.show()

```

Listing 4: 计算温度场分布，并绘制等温线图

4 结果分析

根据代码实现的结果，绘制出不同网格尺寸和松弛因子下的迭代次数的关系图，如图 1所示

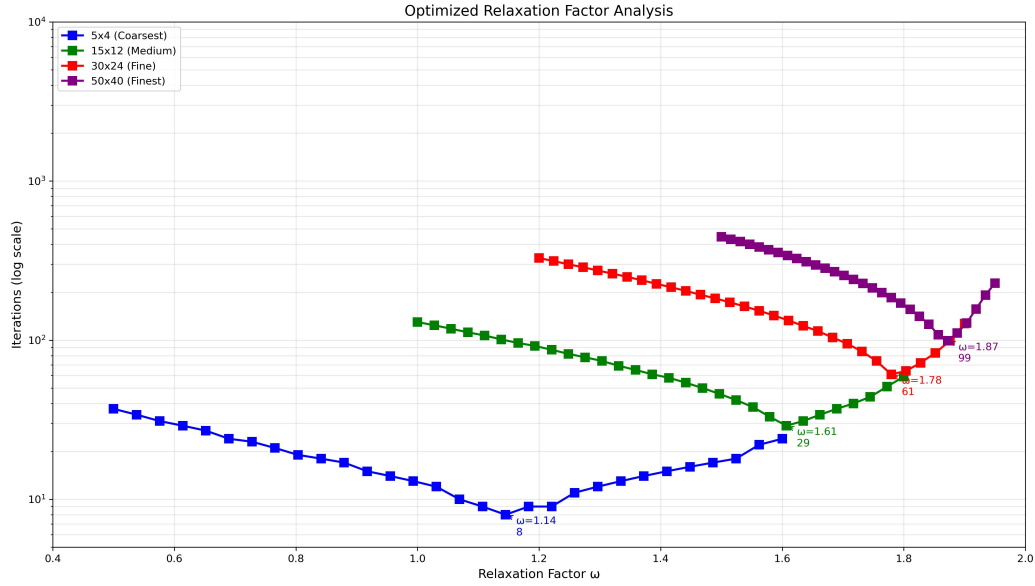


图 1: 不同网格尺寸和松弛因子下的迭代次数关系图

其中，横坐标为松弛因子，纵坐标为迭代次数。可以看出，随着松弛因子的增大，迭代次数逐渐减少，说明 SOR 算法可以有效加快迭代速度；但当松弛因子达到一定值后，迭代次数反而增加。这是因为过大的松弛因子会导致数值不稳定，从而增加迭代次数。

蓝色线为网格数为 5×4 ，绿色线为网格数为 15×12 ，红色线为网格数为 30×24 ，紫色线为网格数为 50×40 。可以看出，随着网格数的增大，迭代次数逐渐增加。这是因为网格数越大，计算量越大，从而增加了迭代次数。

还可以发现，随着网格数的增大，最佳松弛因子逐渐增大。

再绘制出等温线图，如图 2所示：

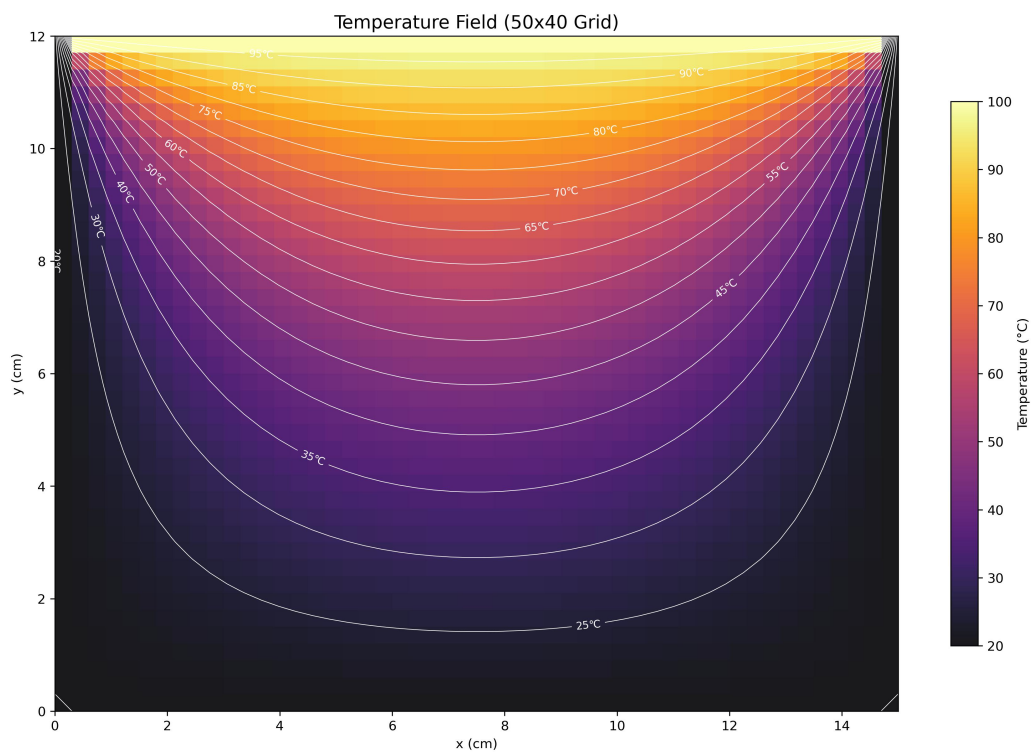


图 2: 等温线图

5 AI 工具使用说明表

AI 名称	生成代码功能	使用内容
Copilot	latex 格式框架	figure 参数调整、图片插入
Deepseek	python 绘图调整	83-104、106-118 行图绘制的具体参数调整
Deepseek	gitignore 文件忽略	全由 ai 添加